ZCAS UNIVERSITY

SCHOOL OF COMPUTING, TECHNOLOGY & APPLIED SCIENCES

CAS5811 DATA SCIENCE ALGORITHMS

FINAL EXAMINATION

27TH MAY 2026

14:00 – 17:00

TIME ALLOWED : WRITING – THREE HOURS

READING – 5 MINUTES

INSTRUCTIONS:

1. Section A: This section is **compulsory** and must be attempted.
2. Sections B: Answer **TWO (2)** questions from this section.
3. This examination paper carries a total of **100 marks.**
4. Candidates must **not turn this page** until the invigilator tells them to do so.

SECTION A: *(COMPULSORY)*

Question 1 : Regression Metrics (25 marks)

(a) Explain the following regression evaluation metrics. For each, provide the mathematical equation and briefly interpret what it measures.

i. Mean Absolute Error (MAE) **[3 marks]**
ii. Mean Squared Error (MSE) **[3 marks]**
iii. Root Mean Squared Error (RMSE) **[3 marks]**
iv. R-squared (R^2) **[3 marks]**

(b) The table below shows actual values (y_i) and predicted values ($\hat{y}_i$) from a regression model for 6 data points.

Observation	Actual (y_i)	Predicted ($\hat{y}_i$)
1	10	12
2	15	14
3	20	18
4	25	27
5	30	29
6	35	36

Using the data above, compute the following. Show all steps and intermediate values.

i. Mean Absolute Error (MAE) **[3 marks]**
ii. Mean Squared Error (MSE) **[3 marks]**
iii. Root Mean Squared Error (RMSE) **[3 marks]**
iv. R-squared (R^2) **[3 marks]**

Question 2 : Big O Notation (25 marks)

(a) Define Big O notation and use precise mathematical language. **[5 marks]**

(b) For each of the following equations, prove the time complexity using the definition of Big O.

i. $f(n) = 5n^3 + 2n^2 + 10$ **[5 marks]**
ii. $f(n) = 3n \log_2 n + 7n + 2$ **[5 marks]**
iii. $f(n) = 2^n + n^{100}$ **[5 marks]**
iv. $f(n) = 15n + 4n\log_2 n + 200$ **[5 marks]**

For each proof, clearly state your chosen **c** and $\mathbf{n_0}$, and provide a step-by-step inequality argument.

(Sub-Total: 50 Marks)

SECTION B: Attempt any TWO (2) questions in this section

Question 3: The Stack Data Structure (25 marks)

Scenario: You are building a feature engineering pipeline for a time series forecasting model. As you preprocess the data, you apply sequential transformations: first impute missing values, then normalize, then create lag features, then apply a rolling window. Sometimes you need to undo the most recent transformation (e.g., to test a different lag value) or revert to a previous state. A stack is ideal for implementing an "undo" feature in this transformation pipeline because transformations are applied in sequence and undone in reverse order.

(a) Explain how a stack, with its Last-In-First-Out (LIFO) behavior, provides the correct functionality for undoing transformations in a feature engineering pipeline. In your explanation, describe the core operations of a stack (push and pop) and why pushing each transformation onto the stack and popping on undo allows you to revert the most recent operation first.

[8 marks]

(b) Suppose your pipeline applies the following four transformations in order to a dataset:
1. ImputeMedian → 2. StandardScale → 3. CreateLagFeatures → 4. RollingWindowSmooth

You decide to undo the last two transformations (RollingWindowSmooth, then CreateLagFeatures).
Trace step-by-step how a stack would manage this undo process. For each step:

- State whether you are pushing or popping.
- Show the contents of the stack after each undo.
- Explain which transformation is currently at the top of the stack after each operation.
- At the end, state which transformation would be undone if the user clicked "undo" one more time.

[8 marks]

(c) Implement a TransformationStack class in Python with the following methods:

- push(transformation_name, data) – stores the name of the transformation and a copy of the dataset before applying the transformation
- pop() – removes and returns the most recent dataset state (raise an exception if empty)
- peek() – returns the most recent dataset state without removing it
- is_empty() – returns True if the stack has no saved states

Then, write a short example demonstrating:

- Applying two transformations (e.g., "ImputeMedian" and "StandardScale")
- Pushing dataset states onto the stack before each transformation
- Using pop() to undo the last transformation and restore the previous dataset state

[9 marks]

(Sub-Total: 25 marks)

Question 4 : Search (25 marks)

Binary search is a fundamental algorithm in computer science and data science. It is used to efficiently locate a target value within a sorted array. Understanding binary search is essential for optimizing search operations, especially when working with large, ordered datasets.

(a) Define the Binary Search algorithm and explain how it works.
[2 marks]

(b) Describe the step-by-step process of searching for a target value in a sorted array using Binary Search. Assume the array is sorted in ascending order.
[6 marks]

(c) Write the pseudocode for the Binary Search algorithm (iterative version).
[5 marks]

(d) Provide the Python implementation of the Binary Search algorithm (iterative version). Ensure that the code is correct, well-commented, and includes an example usage.
[7 marks]

(e) Discuss the practical scenarios or use cases where Binary Search is most suitable. Your discussion should include an explanation of why Binary Search is efficient for large datasets compared to linear search, and mention any limitations (e.g., requirement of sorted data).
[5 marks]

(Sub-Total:25 Marks)

Question 5: Sorting (25 marks)

Sorting algorithms play a pivotal role in data science by organizing data in a specific order, which is crucial for optimizing the efficiency of various tasks in data analysis, machine learning, and algorithm design. They are fundamental to many operations that data scientists perform, from data pre-processing to optimizing the performance of more complex algorithms. Below are key reasons why sorting algorithms are important in data science

a) Define the Bubble Sort algorithm and explain how it works. **[2 Marks]**
b) Describe the step-by-step process of sorting an array using Bubble Sort. **[6 Marks]**
c) Write the pseudocode for the Bubble Sort algorithm. **[5 Marks]**
d) Provide the Python implementation of the Bubble Sort algorithm, ensuring that the code is correct and well-commented. **[7 Marks]**
e) Discuss the practical scenarios or use cases where Bubble Sort is most suitable. Your discussion should include explanations why or why not Bubble Sort is recommended for large datasets, considering its time complexity. **[5 Marks]**

(Sub-total: 25 marks)

END OF EXAMINATION